

運算式合法性檢測

截止日期：2026/06/24 12:00

題目說明

語法解析 (Syntax Parsing) 是編譯器設計與資料結構領域之重要基礎。本作業要求實作一程式，判斷由變數、運算子與括號組成之字串，是否為合法之運算式。

令 $\Sigma = \{A, B, \dots, Z\}$ 為可用之變數集合。一組合法的運算式集合 S 滿足以下定義：

1. 若變數 $x \in \Sigma$ ，則 $x \in S$ 。
2. 若運算式 $E \in S$ ，則 $(E) \in S$ 、 $[E] \in S$ 與 $\{E\} \in S$ 。
3. 若運算式 $E_1, E_2 \in S$ ，則 $E_1 + E_2 \in S$ 與 $E_1 * E_2 \in S$ 。
4. 集合 S 僅包含由上述規則經有限次推導所產生之字串。

若以上下文無關文法 (Context-Free Grammar, CFG) 表示，該合法運算式 E 之生成規則 (Production Rules) 可寫成：

$$E \rightarrow x \mid (E) \mid [E] \mid \{E\} \mid E + E \mid E * E$$

(其中 $x \in \Sigma$)

根據上述定義，例如 A 、 $(A+B)$ 為合法；而 $()$ 、 AB 、 $+A$ 則皆為不合法。

若輸入字串屬於集合 S ，請輸出 `true`，並額外輸出三種括號各自的配對組數；否則，請直接輸出 `false`。

實作規範

1. 程式將於 Linux 環境進行編譯與評測。
2. 程式需支援連續輸入多筆測試資料，每一行字串為一筆測資。
3. 測資保證每行字串長度 L 介於 $1 \leq L \leq 200$ 之間 (不含換行)，且僅由大寫字母 $A-Z$ 、加號 $+$ 、乘號 $*$ 與三種括號 $()$ 、 $\{\}$ 、 $[]$ 所組成。
4. 讀取直到 EOF (End of File) 為止即結束程式。
5. 針對每組輸入，請依據合法性進行對應輸出，每筆輸出的結果需獨立換行。
6. 請提交單一.zip 壓縮檔，內容須包含程式原始碼檔案 (*.c 或 *.cpp)。

範例測試

範例輸入

```
(A+B)*C
[A+B]*{C+D}
A
AB
()
A+*B
(A+)B
[(A+B]
```

範例輸出

```
true ():1 []:0 {}:0
true ():0 []:1 {}:1
true ():0 []:0 {}:0
false
false
false
false
false
```

測試指引

本作業隨附 input.txt 與 output.txt 作為**公開測資**。請利用標準輸入/輸出重導向來驗證程式正確性，以下指令僅供參考：

Linux / macOS 環境

```
gcc -o main main.c
./main < input.txt
```

Windows 環境 (PowerShell / CMD)

```
gcc -o main.exe main.c
type input.txt | .\main.exe
```

參考資料

- 【Data Structure】Day5: 堆疊 (Stack) <https://ithelp.ithome.com.tw/articles/10349112>
- Context-free grammar https://en.wikipedia.org/wiki/Context-free_grammar